

SHARIF UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

PROJECT ASSIGNMENT

Linux Process Telemetry and Unix-Domain IPC Services

INSTRUCTOR / INSTRUCTORS
Dr Iman Gholampour

ACADEMIC CONTEXT
Embedded Systems

Contents

1 Theory questions 2

2 Practical exercise 1: local request/response service 2

3 Practical exercise 2: procwatch system monitor 2

1 Theory questions

1. Explain the roles of bootloader, Linux kernel, root filesystem, C library, init/systemd, and user applications. Describe how startup progresses between these components.
2. Explain why a binary built with the laptop's native g++ on x86 cannot normally execute on an ARM board. Define native/cross compilation, host, and target. Identify toolchain components and their role in producing a target executable. Explain the importance of Make and CMake as a project grows.
3. Explain /bin, /etc, /lib, /dev, /proc, /sys, and /var. Distinguish virtual proc/sys interfaces from ordinary stored files. Compare ext4, SquashFS, and tmpfs for embedded use, including power-loss behavior and flash-write wear.
4. Explain why Device Tree describes hardware and what information it contains. Define Device Tree overlays and give a specific example of enabling a peripheral without rebuilding kernel code.
5. Explain standard Linux latency/jitter sources and difficulties in hard real-time guarantees. Compare PREEMPT_RT with separating time-critical work onto a microcontroller/RTOS while Linux handles less strict functions. Give advantages, limits, tradeoffs, and appropriate use cases.

2 Practical exercise 1: local request/response service

Write a Linux C++ server and client communicating through a pathname Unix-domain socket. The server creates, binds, and listens at a selected path, accepts clients, responds, and remains available for successive connections. Define a simple text protocol with one command returning the server's current date/time and one echoing client-supplied text. The client takes command and arguments from the terminal, connects, sends a request, receives and prints the response, then exits. Supply a Makefile building both applications. Submit source and Makefile and include screenshots of the server and client in separate terminals exercising both commands.

3 Practical exercise 2: procwatch system monitor

Implement procwatch in C++ using Linux /proc. Organize src and include directories and provide both Makefile and CMakeLists.txt. Select interactive or background mode with the first command-line argument. Interactive mode displays live system and process information; background mode remains in the foreground under systemd and writes periodic output to the journal.

3.1 System information

Display kernel version, logical CPU count, aggregate CPU utilization, memory total/used/free/available, and load average in a readable periodically refreshed table, for example every two seconds. Calculate CPU use from differences between consecutive /proc/stat samples. Explain the counters, their meanings, and why one cumulative sample cannot measure interval utilization. Generate one-core load with stress-ng or a busy loop and compare with top/htop.

3.2 Top-N processes

Enumerate numeric /proc directories. Obtain PID and process name. Measure CPU activity from the difference of utime+stime in successive /proc/[pid]/stat samples. Conversion to percentage is

optional, but explain its denominator. Display RSS in specified units, sort by CPU activity, and show the highest N entries, default N=5. Each row includes PID, name, activity, and RSS. Handle common races such as a process exiting between enumeration and reading without crashing.

3.3 systemd operation

In background mode, read at least interval and N from a configuration file supplied by argument. Record top-N data every interval seconds. Handle SIGTERM as an orderly stop, log completion, and exit with code zero. Provide a deliberate error configuration that exits nonzero to test restart. Supply procwatch.service, enable startup at boot, and restart only on failure after approximately five seconds. Submit source, headers, both build descriptions, service, sample configuration, and a short README. The report must preserve build screenshots for Make and CMake, interactive output and CPU-load comparison, systemctl status, journalctl output, graceful SIGTERM termination, and restart following the intentional fault.